CyberArk Conjur Open Source Lab

Technical Build Record and Configuration State

Secrets management deployment, application onboarding patterns, and credential lifecycle operations on WSL2 / Docker

Document scope

Engineering reference: what was deployed, the exact configuration at each stage, and the resulting state of the environment. Problems are recorded as symptom, root cause, and fix rather than as a session narrative. Placeholder values are used for all credentials; no live secret material is reproduced.

Contents

1. Environment and Platform
2. Phase 1 — Vault Deployment
3. Phase 2 — Policy Design (Access Model)
4. Phase 3 — Application Onboarding Patterns
5. Phase 4 — Credential Lifecycle Operations
6. Phase 5 — Kubernetes JWT Authentication
7. Current Configuration State
8. Outstanding Work and Production Considerations

1. Environment and Platform

The lab runs entirely on one Windows laptop: WSL2 (Ubuntu) provides the Linux kernel and host shell, Docker Desktop runs on the WSL2 backend, and the Ubuntu shell is the primary command-line interface. No cloud resources or paid licences are used.

1.1 Platform components

Component	Role	Notes
WSL2 (Ubuntu)	Linux kernel and host shell	Installed via 'wsl --install'. Requires hardware virtualization enabled in firmware.
Docker Desktop	Container engine	WSL2-backed; WSL integration enabled for the Ubuntu distribution.
Python 3 + venv	Runtime for the demonstration applications	Virtual environment 'billing-lab-env' isolates project packages.

1.2 Networking model

Docker Compose creates a private bridge network; every container is resolvable by service name via Docker's embedded DNS. Container-to-container traffic uses internal names and ports (e.g. proxy:443, database:5432); host-to-container traffic uses only the published ports (localhost:8443, localhost:8432).

Recurring consequence: Any process on the WSL host must target localhost with the published port. Any process inside a container targets the internal name and port. Mixing the two is the single most common connection error in this build.

2. Phase 1 — Vault Deployment

Deployed from CyberArk's public conjur-quickstart repository (configuration only; the Conjur image itself is pulled from Docker Hub).

2.1 Container topology

Container	Image	Function
conjur_server	cyberark/conjur	Policy engine, REST API, authentication. No published port for normal operation.
postgres_database	postgres:15	Encrypted backing store for policy and secret data. Published 8432->5432.
nginx_proxy	nginx:latest	TLS-terminating reverse proxy; only externally reachable entry point. Published 8443->443.
conjur_client	cyberark/conjur-cli:9	CLI container for administrative commands.
conjur-quickstart-pgadmin-1	dpage/pgadmin4	Database admin UI. Not used in the build flow. Published 8081->80.
bot_app	cfmanteiga/alpine-bash-curl-jq	Utility container (curl, jq), used once for raw API demonstration.

2.2 Deployment sequence

```
git clone https://github.com/cyberark/conjur-quickstart.git
cd conjur-quickstart && docker compose pull
docker compose run --no-deps --rm conjur data-key generate > data_key
export CONJUR_DATA_KEY="$(< data_key)"
docker compose up -d
```

```
docker compose exec conjur conjurctl account create myConjurAccount > admin_data
```

2.3 CLI initialisation

```
docker compose exec client bash
conjur init -u https://proxy -a myConjurAccount --self-signed
conjur login -i admin
conjur list
```

2.4 Problems encountered

Symptom	Root cause	Fix
conjur_server entered a restart loop after account creation	data-key generate was run without the subcommand; the CLI's own help text was redirected into the data_key file and read back in as the key	Rerun with the correct subcommand, restart the stack
502 Bad Gateway from nginx after conjur_server was recreated	nginx resolves its upstream once at worker start and does not re-resolve; it kept dialing the container's old IP	docker compose restart proxy. Recurs after any restart that reassigns conjur_server's IP.
conjur list failed with "credential not found" despite a successful curl authenticate call	curl returns a valid token but does not write to .netrc; only conjur login does	Run the actual login command

3. Phase 2 — Policy Design (Access Model)

Access control is declarative YAML policy. Three application branches were created under 'prod', each scoped by a five-question framework (real function, systems reached and how each is authenticated to, per-secret access level, accountable owner, blast-radius proportionality) rather than copied from a template.

3.1 Policy constructs used

Tag	Meaning
!policy	A branch in the resource tree; nesting builds the path (prod/billing-app).
!variable	A secret slot. Created empty; value set separately.
!host	A machine identity, authenticated with an API key.
!permit / !deny	Grant or remove a role's privileges on a resource (read, execute, update).
!delete	Removes a resource. Only processed by policy update, never policy load.

3.2 Application branches (final state)

Branch	Identity	Secrets	Privileges	Owner
prod/billing-app	billing-app-01	db/password, api/stripe-key, db/host, db/port, db/username	read, execute	(unset)
prod/operations-app	operations-app-01	powerbi-connection-string	read, execute	Khuli Shaka
prod/procurement-app	procurement-app-01	vendor-api-key	read, execute	Alan Simba

3.3 Example policy (billing branch)

```
- !policy
  id: prod
  body:
```

```
    - !policy
      id: billing-app
      body:
        - &variables
          - !variable db/password
          - !variable api/stripe-key
        - !host billing-app-01
        - !permit
          role: !host billing-app-01
          privileges: [ read, execute ]
          resource: *variables
```

3.4 Correcting the copied branches

Operations and procurement were initially created carrying billing's two secrets, over-provisioning both. Removal requires !deny plus !delete, applied with conjur policy update (PATCH); policy load (POST) rejects deletion statements outright.

```
# cleanup.yml (excerpt)
- !deny
  role: !host operations-app-01
  privileges: [ read, execute ]
  resource: !variable prod/operations-app/api/stripe-key
- !delete
  record: !variable prod/operations-app/api/stripe-key
```

Correctly scoped replacements were then applied with an ordinary policy load. The audit log recorded each removal at per-privilege granularity.

Design principle: Access scope is derived from function, not defaulted. Additive changes (load/POST) and destructive changes (update/PATCH) are deliberately separate operations, so an onboarding action can never silently delete.

4. Phase 3 — Application Onboarding Patterns

The same task, an application obtaining billing's database password and writing a row, was implemented three ways, progressively removing the application from the secret's path. All three use Conjur's native authenticator; they differ in who performs the exchange and where the secret goes afterward.

4.1 Shared authentication mechanism

```
# 1. Authenticate: API key in, short-lived (~8 min) token out
POST /authn/<account>/<host-id>/authenticate   body: <api key>
# 2. Fetch: token in, secret out
GET  /secrets/<account>/variable/<path>   Authorization: Token token="<token>"
```

Conjur supports pluggable authenticators (authn-oidc, authn-k8s, authn-azure) that change only step 1; step 2 and everything downstream are identical.

4.2 Pattern 3a — Direct API retrieval

The application authenticates and fetches the secret itself (billing_app.py, using CyberArk's conjur-api SDK), then opens its own connection to the database.

Finding	Detail
SDK API differs from its own examples	Client uses a multi-step builder (connection info -> credentials -> auth strategy -> client), not a flat constructor; .get() is async and must be awaited
Bootstrap key hygiene	API key moved from hardcoded to an environment variable; reduces accidental leakage but does not eliminate exposure

4.3 Pattern 3b — Summon injection

Summon is a launcher, not a proxy: it authenticates, fetches the secrets named in secrets.yml, injects them as environment variables into a child process, launches the app, and exits. The app contains no Conjur code.

```
DB_PASSWORD: !var prod/billing-app/db/password
STRIPE_KEY:  !var prod/billing-app/api/stripe-key
```

Finding	Detail
All-or-nothing fetch	A single empty/unreadable variable in secrets.yml aborts the entire launch (found when api/stripe-key had no value set)
Injected variables are ephemeral	Confirmed: echo $DB_PASSWORD in the host shell after the app exits returns empty

Bootstrap credential: Summon removes application secrets from the app, but billing-app-01's own API key still sits in the host .netrc (chmod 600). Middle rung, not the top.

4.4 Pattern 3c — Secretless Broker

The broker sits permanently in the connection path. The application connects to a local port believing it's a plain database; the broker authenticates to Conjur, fetches the full connection detail set, opens the real authenticated connection, and splices the two. The application never receives a credential.

```
credentials:
  host: { from: conjur, get: prod/billing-app/db/host }
  port: { from: conjur, get: prod/billing-app/db/port }
  username: { from: conjur, get: prod/billing-app/db/username }
  password: { from: conjur, get: prod/billing-app/db/password }
  sslmode: disable   # literal; broker-to-DB hop, lab only
```

Symptom	Root cause	Fix
App connection refused: "backend does not allow SSL connections"	Broker's back-side connection to Postgres attempted SSL by default; stock postgres:15 has none configured	sslmode: disable added to secretless.yml credentials
Identical opaque errors regardless of which credential failed	Broker does not expose which secret or hop failed to the client	Diagnosis requires broker logs; distinguishes "failed on retrieve credentials" (Conjur-side) from "failed on connect" (DB-side)
Broker lost trust in Conjur after a stack restart	Self-signed certificate regenerated; broker still held the cert from its creation-time environment variable	Re-pin client cert, copy the fresh cert out, recreate the broker container

Two properties established by testing: credentials resolve per connection, not once at startup (rotation needs no broker restart); revoking any one of the four required variables breaks the connection identically.

4.5 Onboarding progression

Pattern	Who authenticates	Secret path	App holds secret?
3a Direct API	The application	Conjur -> app -> database	Yes, in memory
3b Summon	Summon (launcher)	Conjur -> Summon -> env var -> app -> database	Briefly, as env var
3c Secretless	The broker	Conjur -> broker -> database	No, never

5. Phase 4 — Credential Lifecycle Operations

5.1 Manual rotation and the drift window

```
docker compose exec database psql -U postgres \
  -c "ALTER ROLE billing_app WITH PASSWORD 'RotatedPassword456';"
# all three apps fail here until the vault is updated to match
conjur variable set -i prod/billing-app/db/password -v 'RotatedPassword456'
```

Tested assumption disproved: expected the Secretless broker to require a restart to pick up the new value. It did not; broker log confirmed resolution *on handle connection*, i.e. per connection. All three patterns self-heal on next use. The actual availability risk is the vault-to-target drift window (~60s observed), not credential caching.

5.2 Automated rotation

A scripted rotation closes the drift window by performing both sides in immediate succession, using a dedicated, narrowly scoped rotation identity rather than the Postgres superuser or the billing identity.

Object	Scope
Postgres role 'rotator'	CREATEROLE (can alter other roles' passwords); not superuser
prod/db-admin (rotation-job-01)	read/execute on rotator-password
Grant on prod/billing-app/db/password	update only, no read/execute — can change the password without being able to read it

rotate_billing_password.sh (set -euo pipefail) authenticates as rotation-job-01, fetches the rotator credential, generates a 24-character random password, applies ALTER ROLE, writes the new value back to the vault. Observed: all three log stages complete within the same second (no drift). Pipeline-abort confirmed twice on bad input; the script aborted before touching the vault, leaving no partial state.

Scheduling status: Crontab entry (0 2,14 * * *) documented as the intended schedule, not enabled: WSL2 does not run cron persistently across Windows restarts without extra configuration. On-demand execution is the working evidence.

Residual credential: rotation-job-01's own API key cannot be vaulted, since it's the credential used to reach the vault. Stored in ~/.rotation_env, chmod 600. Compensating controls: file permissions, scheduled key rotation, an audit rule for off-schedule authentication.

5.3 Connection-detail variables (added for Secretless)

db/host (`database`), db/port (5432, internal), and db/username (`billing_app`) added to the billing branch and granted to billing-app-01. Internal Docker values, since the broker connects from inside the network.

5.4 Revocation

Revoking billing-app-01's read/execute on db/password (!deny + policy update) caused all three applications to fail immediately, with no restart and no cached-credential grace period, confirming per-fetch resolution across all patterns. Revocation is surgical: only the named identity's grant is removed.

Finding	Detail
404, not 403, on permission denial	Deliberate: prevents resource enumeration by not confirming a path exists to a caller not permitted to see it. Consequence: ambiguous between a path typo and a missing grant.
Secretless gives no app-side signal of which variable failed	Revoking db/username produced an identical opaque error to revoking db/password. Only the broker log names the specific variable.

6. Phase 5 — Kubernetes JWT Authentication (Platform-Native Identity)

Eliminates the bootstrap credential. In every earlier pattern, something held a static Conjur API key. Here, a Kubernetes pod authenticates using its own service account token, a short-lived JWT the cluster issues and mounts automatically, never stored anywhere static.

6.1 Supporting cluster

Single-node cluster created with `kind` (v0.27.0, node image v1.32.2), on a separate Docker network (`kind`, 172.19.0.0/16) from the Conjur stack (172.18.0.0/16). Attached to the Conjur network with `docker network connect` so pods can resolve `proxy`.

> **Fragility:** docker network connect does not survive stack restarts; must be re-established alongside the nginx upstream refresh and certificate re-pin each session.

6.2 Authenticator configuration

```
- !policy
  id: conjur/authn-jwt/k8s-cluster
  body:
    - !webservice
    - !variable public-keys
    - !variable issuer
    - !variable token-app-property
    - !variable identity-path
    - !variable audience
    - !group consumers
    - !permit
      role: !group consumers
      privilege: [ read, authenticate ]
      resource: !webservice

- !policy
  id: prod
  body:
    - !policy
      id: k8s-apps
      body:
        - !host
          id: system:serviceaccount:billing-app:billing-sa
          annotations:
            authn-jwt/k8s-cluster/sub: system:serviceaccount:billing-app:billing-sa
```

Static JWKS approach used (public key embedded directly in Conjur), since Conjur cannot reach the kind API server's JWKS endpoint across the separate networks. Enabled by exporting CONJUR_AUTHENTICATORS="authn,authn-jwt/k8s-cluster" and recreating the conjur container.

6.3 Pod authentication flow

```
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
CONJUR_TOKEN=$(curl -sk -X POST \
  "https://proxy/authn-jwt/k8s-cluster/myConjurAccount/authenticate" \
  --data-urlencode "jwt=$TOKEN" | base64 | tr -d '\n')
curl -sk -H "Authorization: Token token=\"$CONJUR_TOKEN\"" \
  "https://proxy/secrets/myConjurAccount/variable/prod%2Fbilling-app%2Fdb%2Fpassword"
```

Successfully retrieved the 24-character database password. No API key exists in the pod spec, environment, mounted secret, or .netrc.

6.4 Problem encountered

Symptom	Root cause	Fix
401 Unauthorized, empty body, on first authentication attempt	JWT::InvalidAudError — signature and issuer validated; token's audience did not match Conjur's default expectation	Added an audience variable to the authenticator, set to match the token's actual value

Significance: The only pattern with no stored credential of any kind. Demonstrates, as working fact, that the Level 1 bootstrap credential identified in the key custody analysis is eliminable via platform-native identity.

7. Current Configuration State

7.1 Vault objects

Path / object	Type	Notes
myConjurAccount:user:admin	user	Bootstrap admin identity
prod/billing-app/billing-app-01	host	read/execute on all five billing variables
prod/billing-app/db/password	variable	Current value is a 24-char random string from the last rotation
prod/billing-app/api/stripe-key	variable	Fake test value
prod/billing-app/db/host | db/port | db/username	variable	database | 5432 | billing_app
prod/operations-app/operations-app-01	host	read/execute on powerbi-connection-string
prod/procurement-app/procurement-app-01	host	read/execute on vendor-api-key
prod/db-admin/rotation-job-01	host	read/execute on rotator-password; update on billing db/password
prod/break-glass/break-glass-01	host	read/execute on all four app secrets; key rotated after use
conjur/authn-jwt/k8s-cluster	webservice	JWT authenticator; enabled via CONJUR_AUTHENTICATORS
prod/k8s-apps/ system:serviceaccount:billing-app:billing-sa	host	JWT identity; read/execute on billing db/password; no API key

7.2 Running components

Component	State
6 compose containers (conjur, postgres, nginx, client, pgadmin, bot_app)	Running via docker compose
secretless_broker	Standalone container, not in the compose file; does not restart with the stack
kind cluster 'conjur-lab'	Namespace billing-app, service account billing-sa, pod billing-test
billing_app.py / billing_app_summon.py / billing_app_secretless.py	Three working apps on the host
rotate_billing_password.sh + ~/.rotation_env	Working on-demand automated rotation; cron not enabled

7.3 Known fragilities

- Self-signed certificate regeneration on stack restart breaks CLI and broker trust until re-pinned. Recurred multiple times; the single most impactful operational fragility.

- Secretless broker and the kind network attachment are not declarative; both must be re-established manually after a restart, alongside `CONJUR_DATA_KEY` and `CONJUR_AUTHENTICATORS` being re-exported before any conjur container recreate.

8. Outstanding Work and Production Considerations

All six phases are complete. The items below are the remaining gaps between this lab and a production-grade deployment.

8.1 Production Considerations

- Single-instance, self-signed, manually-keyed deployment, intentional for learning. Production adds clustering/HA, CA-issued certificates with defined rotation, external KMS/HSM custody of the master data key, HA Postgres, DR, and a Kubernetes-based Conjur Leader/Follower architecture.
- The bootstrap identity credential applies to the direct-API, Summon, and Secretless patterns. Phase 5 demonstrates its elimination via the JWT authenticator; production extends the same principle to non-Kubernetes workloads via authn-azure / Managed Identity.
- Manual/scripted rotation stands in for CyberArk CPM, which performs the target change and vault update as one automated, scheduled operation via a managed reconciliation account.
- Broker back-side connection uses sslmode=disable; production enables TLS on the database and mTLS between components.
- Audit logs remain in container stdout; production forwards to SIEM with detection rules for break-glass authentication, off-schedule rotation-job activity, repeated 401s, and out-of-window policy changes.
- No formal CIS Docker Benchmark scan or image vulnerability scan performed; Secretless plugin-hash verification not enabled.

End of build record.